

Advanced Deployment of AAP 2.7

Version 1, 2026-07-31

Home

Introduction

Ansible Automation Platform (AAP) on OpenShift Container Platform (OCP) delivers enterprise-grade automation through the AAP Operator. A core feature of this deployment model is Automation Mesh, which enables scalable job distribution across remote execution nodes via peer-to-peer Receptor connections — all managed through the AAP UI without re-running the installer. In this training we will explore how to design and configure Automation Mesh in an operator-based environment, size your deployment using OpenShift Pod resource models, and troubleshoot common issues using OpenShift tooling.

Duration: 60 minutes

Objectives

On completing this course, you will:

- Understand how Automation Mesh works in an operator-based (OCP) deployment;
- Configure Container Groups and remote execution nodes using the AAP UI and operator custom resources;
- Calculate sizing requirements expressed as Pod resource requests and limits;
- Troubleshoot AAP installation and mesh issues using the `oc` CLI and operator conditions.

Prerequisites

This course assumes that you have the following prior experience:

- Foundational knowledge of Linux, Containers, Kubernetes/OpenShift, and Ansible.
- [Basic Deployment of Ansible Automation Platform](#).
- Familiarity with OpenShift concepts: namespaces, pods, service accounts, routes, and RBAC.

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Konstantin Kuminsky (Contributor)
- Amrinder Singh (Content Architect)
- Rutuja Deshmukh (Editor)
- Anna Swicegood (Learning Product Manager)

Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. **Log in** to the [RHDP portal](#).
2. **Search** for the catalog entry: **Advanced Deployment of Ansible Automation Platform 2.7**.



Product Enablement: Advanced Deployment of AAP 2.7

provided by Product Technical Learning

Order



 Save as favorite

Category
Labs

Provider
Product Technical Learning

Product Family
Red Hat Automation

Rating
★★★★☆ (15)

Cost
\$0.16/hr \$\$\$

Estimated provision time
±24 minutes

Uptime ⓘ

100%

Last update
3 hours ago

Last successful provision
29 hours ago

Auto-Stop
6 Hours

Auto-Destroy
48 Hours

Description

Learn about AAP 2.7 advanced deployment topologies, Automation Mesh sizing, and hands-on containerized and operator-based installation across multiple nodes.

Ansible Automation Platform 2.7 enables scalable IT automation across complex environments. This lab pre-installs the AAP operator on OpenShift and provisions three dedicated VMs – an execution node, a containerized install host, and a container execution node – so you can practice real-world multi-node deployment patterns.

Lab Guide

You can access the lab guide though.

- [Red Hat LMS](#) (preferred)
- [Quick Course](#)

What Content Is Covered In The Lab?

- Understanding of Automation Mesh and multi-node topologies
- Calculate the sizing requirement of Ansible Automation Platform
- Operator-based AAP deployment on OpenShift with Automation Mesh.
- Containerized AAP deployment across dedicated VMs with Automation Mesh.
- Troubleshooting Ansible Automation Platform installation issues

Environment Specifications

- **OpenShift SNO Cluster** – Pre-installed AAP operator

3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.

Activity *

- ☐ Customer Facing
- ☐ Partner Facing
- ☒ Practice / Enablement

Purpose *

Trying out a technical solution ▼



Salesforce ID *(Opportunity ID, Campaign ID, CDH Party or Project ID)*

- ☐ Campaign ☐ CDH ☐ Opportunity ☐ Project 

Salesforce ID

 Search



☐ I'll provide the Salesforce ID within 48 hours. 

6. **Review the warning** at the bottom of the form and **check the box** labeled:
“I confirm that I understand the above warnings.”
7. **Click the Order** button to place your lab order.

Important Notes:

- This lab may take approximately **20 minutes** to become ready.
- You will receive an **email with access details** once your lab environment is ready.
- You can also **retrieve lab access** directly from the RHDP portal.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, **click** on the **Services** option in the left-hand menu.
2. **Select your lab** from the listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: <https://demo.redhat.com/>
- RedHat partners: <https://partner.demo.redhat.com/>

Advanced Deployment of Ansible Automation Platform 2.7 on OpenShift

Ansible Automation Platform on OpenShift Container Platform provides scalable, operator-managed IT automation. This course focuses on Automation Mesh in operator-based environments, where the Control Plane runs as pods inside the Kubernetes cluster and remote execution nodes peer to it via Receptor over TLS.

Participants will acquire practical skills in:

- Configuring Automation Mesh for an operator-based (OCP) deployment using Container Groups and the AAP UI.
- Adding remote execution and hop nodes using the `AutomationControllerMeshIngress` custom resource.
- Determining cluster sizing by translating fork and event calculations into Pod resource requests and limits.
- Troubleshooting common installation and mesh issues using `oc` CLI and operator status conditions.

Automation Mesh on OpenShift

Automation Mesh is an overlay network designed to enable the allocation of work among a large and dispersed group of workers known as Execution Nodes. This is achieved by creating peer-to-peer connections between nodes over existing networks. Receptor is the service that manages these connections.

Automation Mesh communications are encrypted with TLS, which allows securing traffic that moves across external networks such as the Internet, and enables AAP to achieve the following:

Key capabilities automation mesh introduces:

Capability	Description
Dynamic cluster capacity	Create, register, group, ungroup, and deregister nodes with minimal downtime
Plane separation	Scale playbook execution capacity independently from control plane capacity
Resilient deployment	Reconfigurable without outage; dynamically re-routes when outages occur
FIPS-compliant connectivity	Bi-directional, multi-hopped mesh communication
TLS encryption	All traffic traversing external networks is encrypted in transit

Automation mesh is useful for:

- Traversing difficult network topologies
- Bringing execution capabilities (the machine running `ansible-playbook`) closer to your target hosts

Control Plane on OpenShift

In an operator-based (OCP) deployment, the Control Plane is comprised of pods running inside the Kubernetes cluster — there are **no Hybrid or Control nodes** as in a VM-based deployment. The AAP Operator manages these pods and runs persistent Ansible Automation Platform services:

- Web server
- Task dispatcher
- Project updates
- Management jobs

The key difference from a VM-based deployment is that the Control Plane local to the Kubernetes cluster is made up of **Container Groups** — containers running on the cluster — rather than Hybrid or Control nodes.

Execution Plane on OpenShift

The Execution Plane consists of execution nodes that execute automation on behalf of the Control Plane and have no control functions. Hop nodes serve to route traffic. Execution Plane nodes only run user-space jobs, and may be geographically separated, with high latency, from the Control Plane.

- **Execution nodes:** Run jobs using `ansible-runner` with `podman` isolation. These are standalone RHEL VMs registered to the automation controller as type `execution` instances. They are only used to run jobs, not dispatch work or handle web requests.
- **Hop nodes:** Similar to a jump host, hop nodes route traffic to other execution nodes. Hop nodes cannot execute automation. They are registered in automation controller as an instance of type `hop`, handling inbound and outbound traffic for otherwise unreachable nodes in different or more strict networks.



In operator-based environments only `execution` and `hop` instance types can be created. Operator-based deployments do **not** support Control or Hybrid nodes.

Container Groups

Container Groups are a special type of Instance Group available in operator-based deployments. They enable automation controller to run jobs in containers provisioned on-demand as pods that exist only for the duration of the playbook run. This is known as the ephemeral execution model and ensures a clean environment for every job run.

Container Groups act as a pool of resources within an OpenShift namespace. You can create instance groups to point to an OpenShift container, using a service account and a bearer token to authenticate to the cluster.



Container groups do not use the capacity algorithm that normal execution nodes use. You set the number of forks at the job template level. The fork setting is passed directly to the container.

Peers

Peer relationships define the node-to-node connections that form the automation mesh. In an operator-based environment, peers are defined through the AAP UI for individual instances — not via an installer inventory file.

When you add an execution or hop node through the UI, you configure whether it peers directly from control nodes (**Peers from control nodes**) or is peered through a hop node.

Now let's move on to the practical implementation of Automation Mesh on OpenShift. In the next section, we will cover how to set up and configure the Control Plane and Execution Plane, as well as how to establish peer relationships between nodes.

Automation Mesh Configuration on OpenShift hands-on Lab

In an operator-based AAP deployment, Automation Mesh is configured entirely through the AAP UI and OpenShift custom resources — not through installer inventory files. This section walks through the remote execution for extending the mesh beyond the cluster.

Lets setup the VM and connect it to Ansible Automation Platform for the execution node:

1. OpenShift Console Access:

Open the OpenShift Console at `{openshift_console_url}` and log in with username `admin` and password `{common_admin_password}`.

- Navigate to **Virtualization** → **VirtualMachines**.
- Select namespace: `aap-lab`
- Click on `aap-lab-execution`

[1.1] | 1.1.png

- Click on the **Open Web Console** button.

[1.2] | 1.2.png

- Use the **Serial** tab for terminal access.

[1.3] | 1.3.png

- Log in to the VM using username `{cloud_init_user}` and password `{common_user_password}`. Copy and paste with CTRL+SHIFT+V in the terminal.

3. Register the system with Red Hat Subscription Management and attach the subscription to the system.

```
sudo subscription-manager register
```

```
username: <YOUR-RHN_LOGIN>  
password: <YOUR-RHN_PASSWORD>
```

```
aap-lab-execution login: lab-user  
Password:  
[lab-user@aap-lab-execution ~]$ sudo subscription-manager register  
Registering to: subscription.rhsm.redhat.com:443/subscription  
Username: rhn-support-amrsingh  
Password:  
The system has been registered with ID: e3990699-0e24-477d-9c63-e7c  
The registered system name is: aap-lab-execution.headless.aap.svc.c
```

- a. Enable the required repository for AAP 2.7:

```
sudo subscription-manager repos --enable ansible-automation-platform-2.7-for-  
rhel-10-x86_64-rpms
```

```
[lab-user@aap-lab-execution ~]$ sudo subscription-manager repos --enable ansible-automation-plat  
Repository 'ansible-automation-platform-2.7-for-rhel-10-x86_64-rpms' is enabled for this system.  
_
```

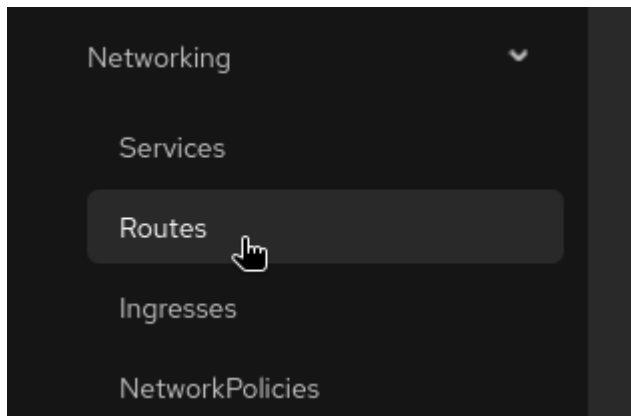
4. Install the required and useful utilities:

```
sudo dnf install -y vim ansible-core
```

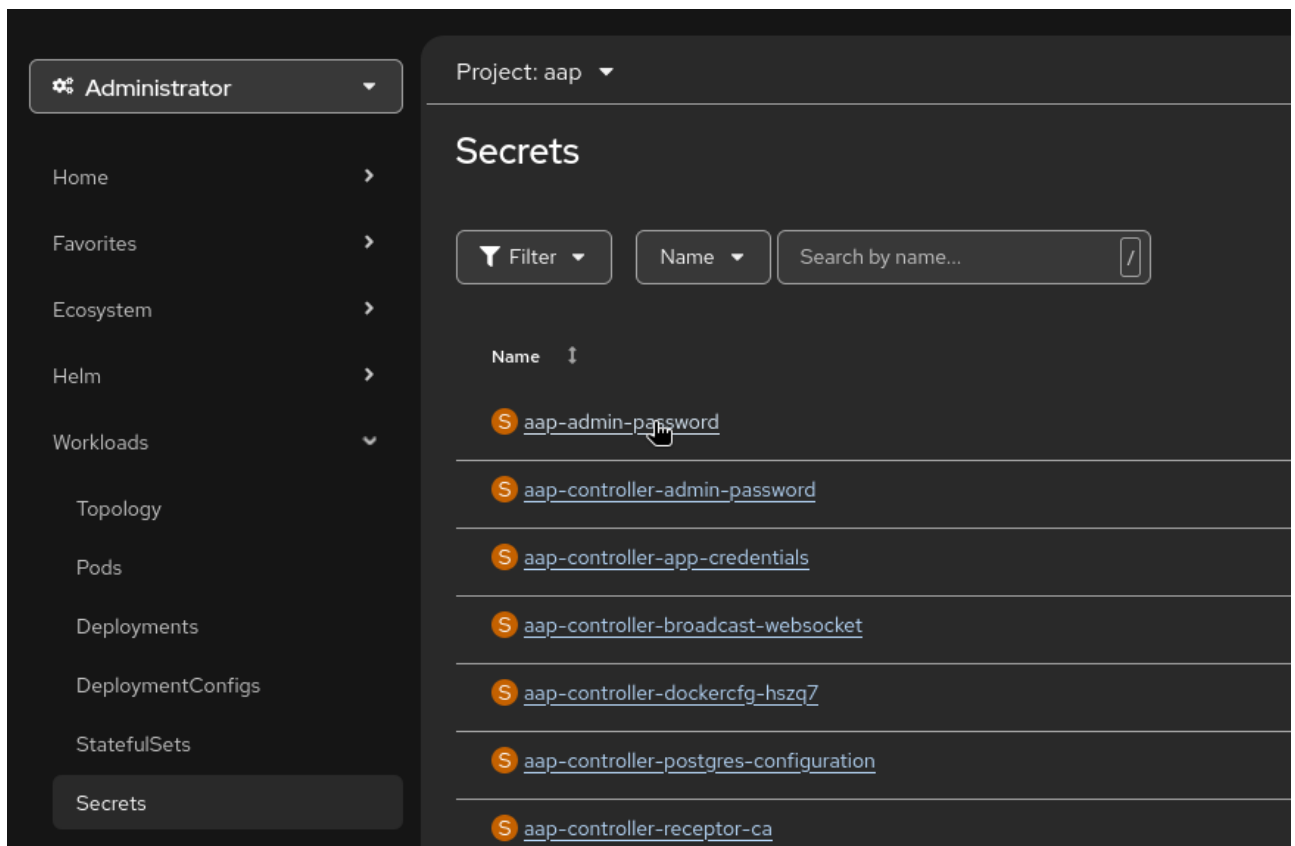
Install Receptor on the Execution Node:

Ansible Automation Platform UI Configuration:

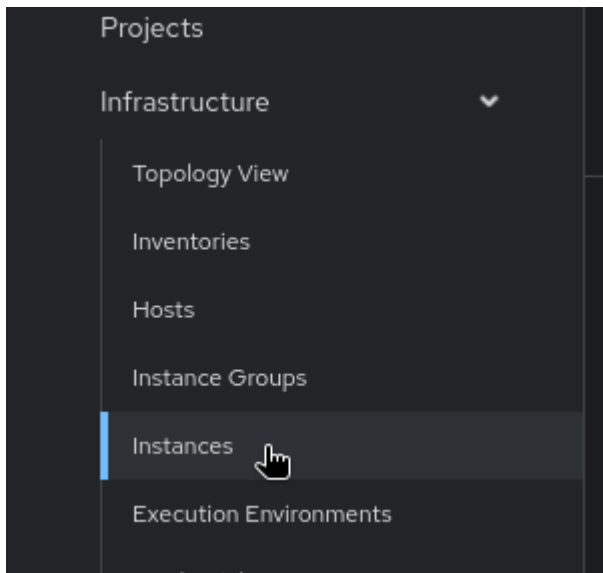
1. Login to the AAP UI for that go to openshift cluster and then go to **Networking > Routes > Location for the aap Service** open the URL in the browser in the new tab.



2. Go to **Workloads > Secrets > aap-admin-password** and copy the password to login to the AAP UI. for this lab you can use this for admin password: `{common_admin_password}`.



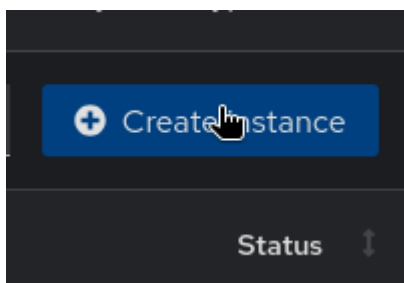
3. Download the Receptor install bundle from the AAP UI and copy it to the execution node. Use the following command to extract the bundle:
 - a. From the navigation panel, select **Automation Execution > Infrastructure > Instances**.

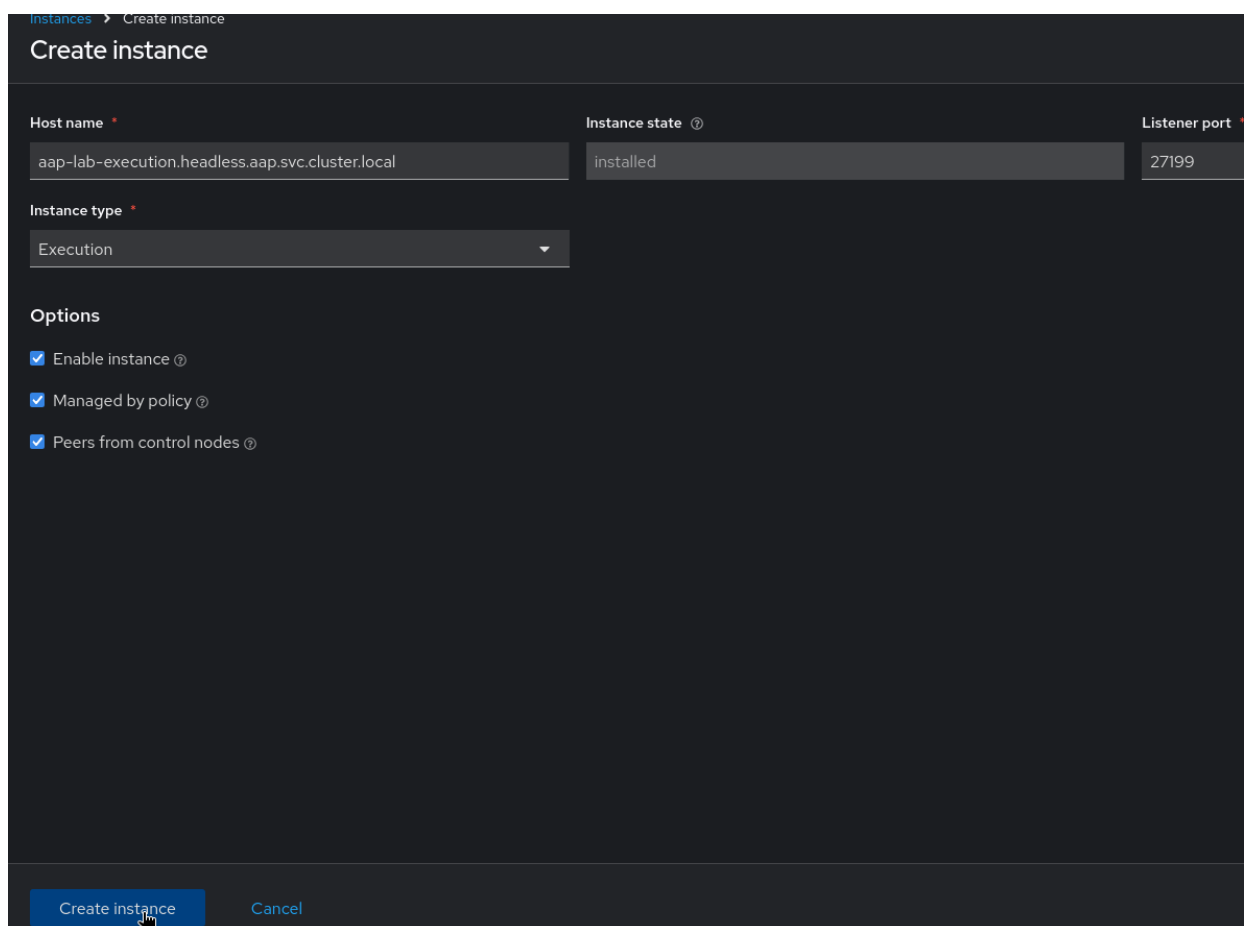
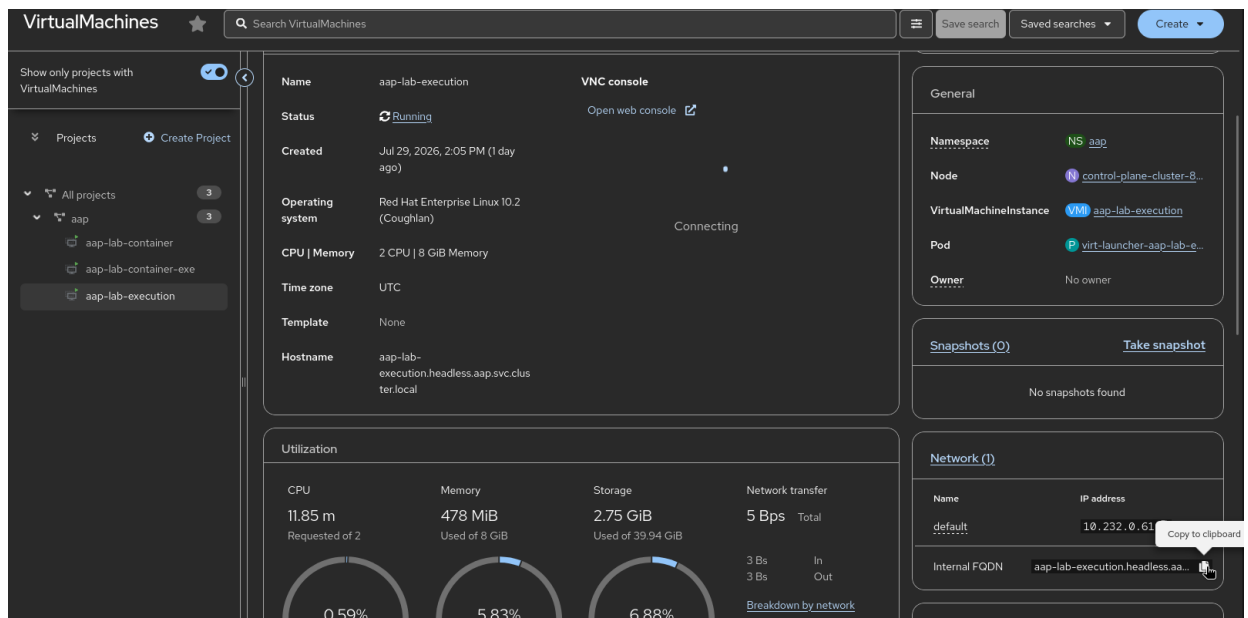


- b. On the **Instances** list page, click **Add instance**. Fill in the following fields:

Field	Description
Host name	Fully qualified domain name (public DNS) or IP address. Use IP if private DNS cannot be resolved from the control cluster.
Description	(Optional) A description for the instance.
Instance state	Auto-populated (Installed); cannot be modified.
Listener port	Port for Receptor to listen on for incoming connections. Default: 27199 .
Instance type	Only execution and hop are supported on OpenShift operator deployments.

- c. Add the VM **aap-lab-execution** FQDN from the Openshift cluster **Virtualization** > **VirtualMachines** > **aap-lab** > **aap-lab-execution** and the port **27199** to the instance and click **Save**.





- d. Copy the URL of the bundle from the AAP UI and download it on the execution node using the following command:

[1.12] | 1.12.png

Setup Receptor on the Execution Node:

1. Download and Extract the bundle on the remote machine. The aap admin password is {common_admin_password}.

```
curl -u admin:<admin_password> --output aap.tar.gz <Paste the URL of the bundle from the AAP UI>
```

```
[lab-user@aap-lab-execution ~]$ curl -u admin:Yj8r5p6lRrhQ --output aap.tar.gz https://aap-aap.apps.cluster-8x2nx.dyn.redhatworkshops.io/api/controller/v2/instances/2/install_bundle/
ut aap.tar.gz https://aap-aap.apps.cluster-8x2nx.dyn.redhatworkshops.io/api/controller/v2/instances/2/install_bundle/
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 5459 100 5459 0 0 8427 0 --:--:-- --:--:-- --:--:-- 8437
[lab-user@aap-lab-execution ~]$ ls
aap.tar.gz
[lab-user@aap-lab-execution ~]$ _
```

```
tar -xzf aap.tar.gz
```

2. Move into the extracted directory and run the install script:

```
cd <extracted directory>
```

3. Edit `inventory.yml`

```
vim inventory.yml
```

```
all:
  hosts:
    remote-execution:
      ansible_host: <this-should-be-the-mesh-node-host-name> # change to the mesh
node host name if its not
      ansible_user: lab-user # this should have the username of the remote
execution node
      # ansible_ssh_private_key_file: ~/.ssh/<id_rsa>
```

```
---
all:
  hosts:
    remote-execution:
      ansible_host: aap-lab-execution.headless.aap.svc.cluster.local
      ansible_user: lab-user # user provided
      # ansible_ssh_private_key_file: ~/.ssh/id_rsa
~
```

4. Install the `ansible.receptor` collection:

```
sudo ansible-galaxy install -r requirements.yml
```

```
[lab-user@aap-sudo ansible-galaxy install -r requirements.ymlc.cluster.local_install_bundle]$ sudo ansible-galaxy install -r requirements.yml
Starting galaxy collection install process
Process install dependency map
Starting collection install process
Downloading https://galaxy.ansible.com/api/v3/plugin/ansible/content/published/collections/artifacts/ansible-receptor-2.0.3.tar.gz to /root/.ans
Installing 'ansible.receptor:2.0.3' to '/root/.ansible/collections/ansible_collections/ansible/receptor'
ansible.receptor:2.0.3 was installed successfully
```

Open port 27199 for Receptor communication on the Firewall of the execution node and the communication should be allowed bi-directionally between the execution node and the control plane nodes.

- Run the installation playbook and provide the password: `{common_admin_password}` for the execution node when prompted. This will install Receptor on the execution node and configure it to connect to the control plane nodes.:

```
sudo ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i inventory.yml
install_receptor.yml -k
```

```
skipping: [remote-execution]

RUNNING HANDLER [ansible.receptor.setup : Reload Receptor] *****
skipping: [remote-execution]

PLAY RECAP *****
remote-execution      : ok=45   changed=19   unreachable=0   failed=0   skipped=9   rescued=0   ignored=0
```

- Once the installation is complete successfully. Proceed to the next step to verify the connection between the execution node and the control plane nodes by clicking on the **Run Health Check** button in the AAP UI and soon the status will show **Ready** for the Execution node.

The screenshot shows the 'Details' tab for the node 'aap-lab-execution.headless.aap.svc.cluster.local'. The status is 'Installed'. A 'Run health check' button is located in the top right corner. The node's listener port is 27199, and its policy type is 'Auto'. The node is enabled, as indicated by the toggle switch.

The screenshot shows the 'Instances' page in the AAP UI. It lists two instances: 'aap-controller-task-86487c9b88-gfk6t' (Control type, Ready status) and 'aap-lab-execution.headless.aap.svc.cluster.local' (Execution type, Ready status). The 'aap-lab-execution.headless.aap.svc.cluster.local' node is highlighted, showing its capacity adjustment (76 forks) and used capacity (0%).

Let run a JOB on the execution node to verify if it will get executed. The job might be failing due to EE but the execution node should be the **aap-lab-execution** node that we have assigned.

Ansible Automation Platform 2.7 Job run on execution nodes:

Part 1: Create a Instance Group:

- Navigate to **Automation Execution** > **Infrastructure** > **Instance Groups**.
- Click **Create group** and select **Create instance Groups**.
- Enter **IG** as the name and then go to the **Instances** tab and add the Instance you created in the previous step. Click **Save**.

Overview

Automation Execution Automation Controller

Jobs

Templates

Schedules

Projects

Infrastructure

Topology View

Inventories

Hosts

Instance Groups

Instances

Execution Environments

Instance Groups ?

An instance group provides the ability to group instances in a clustered environment.

☐ Search Enter search

+ Create group

Create container group

Create instance group

Name	Type	
☐ controlplane	Instance group	0
☐ default	Container group	0

Name *

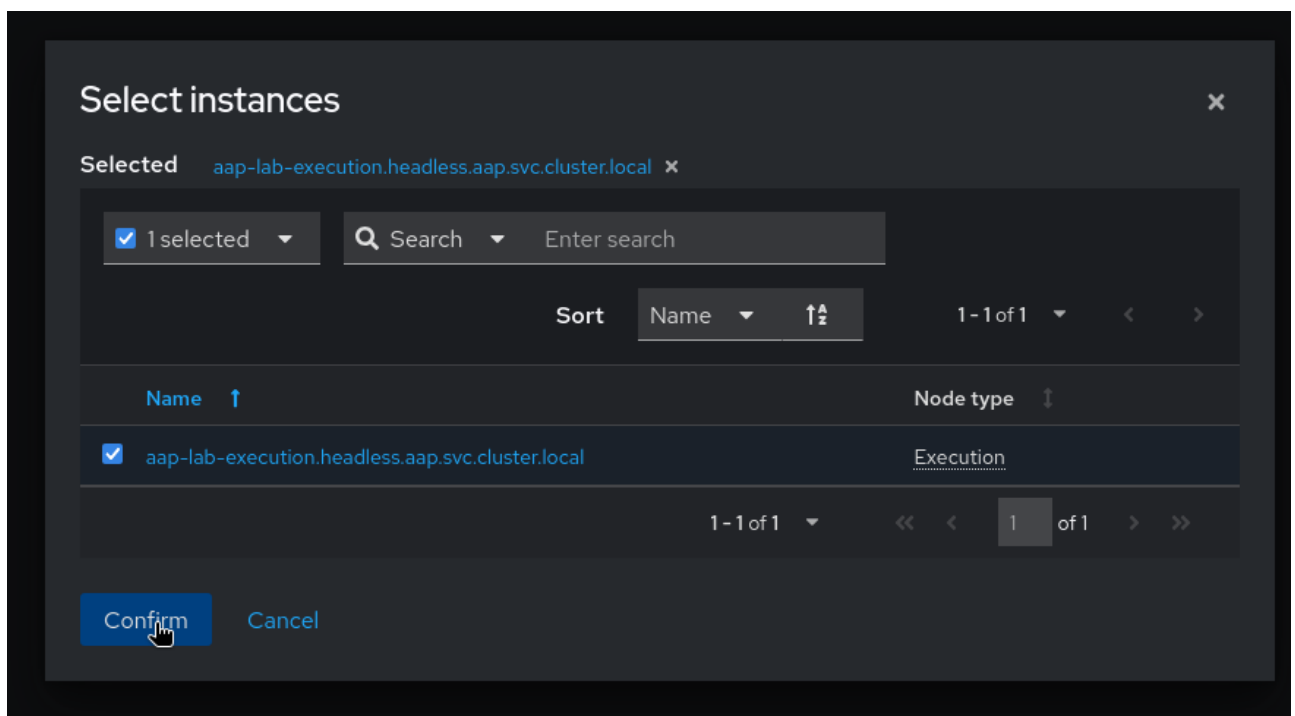
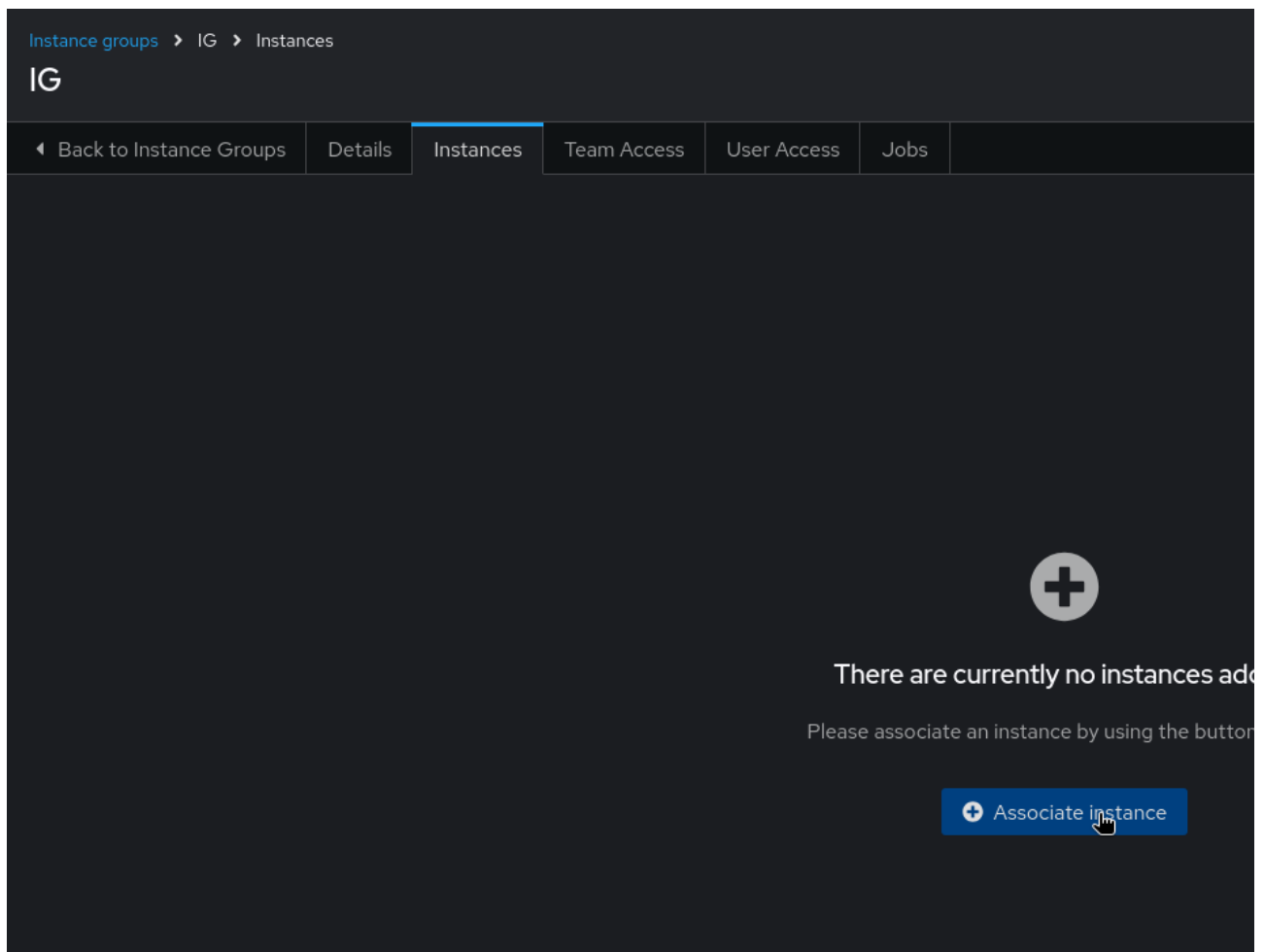
IG

Max concurrent jobs ?

0

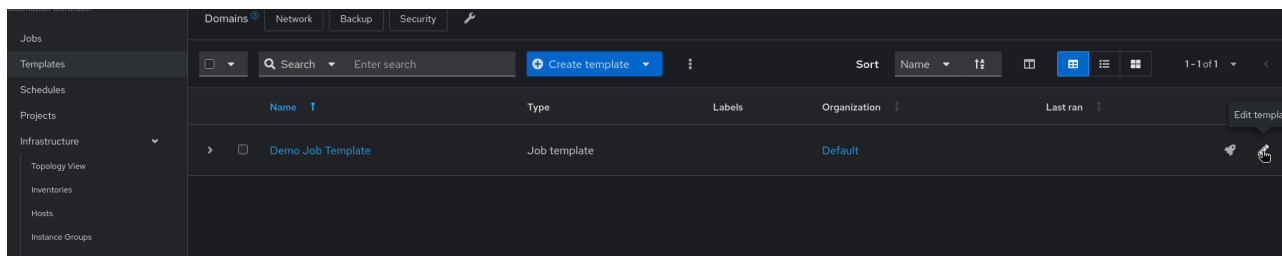
Create instance group Cancel

- Assign the Instance to this Instance Group and click on `aap-lab-execution.headless.aap.svc.cluster.local` and then Confirm the selection.

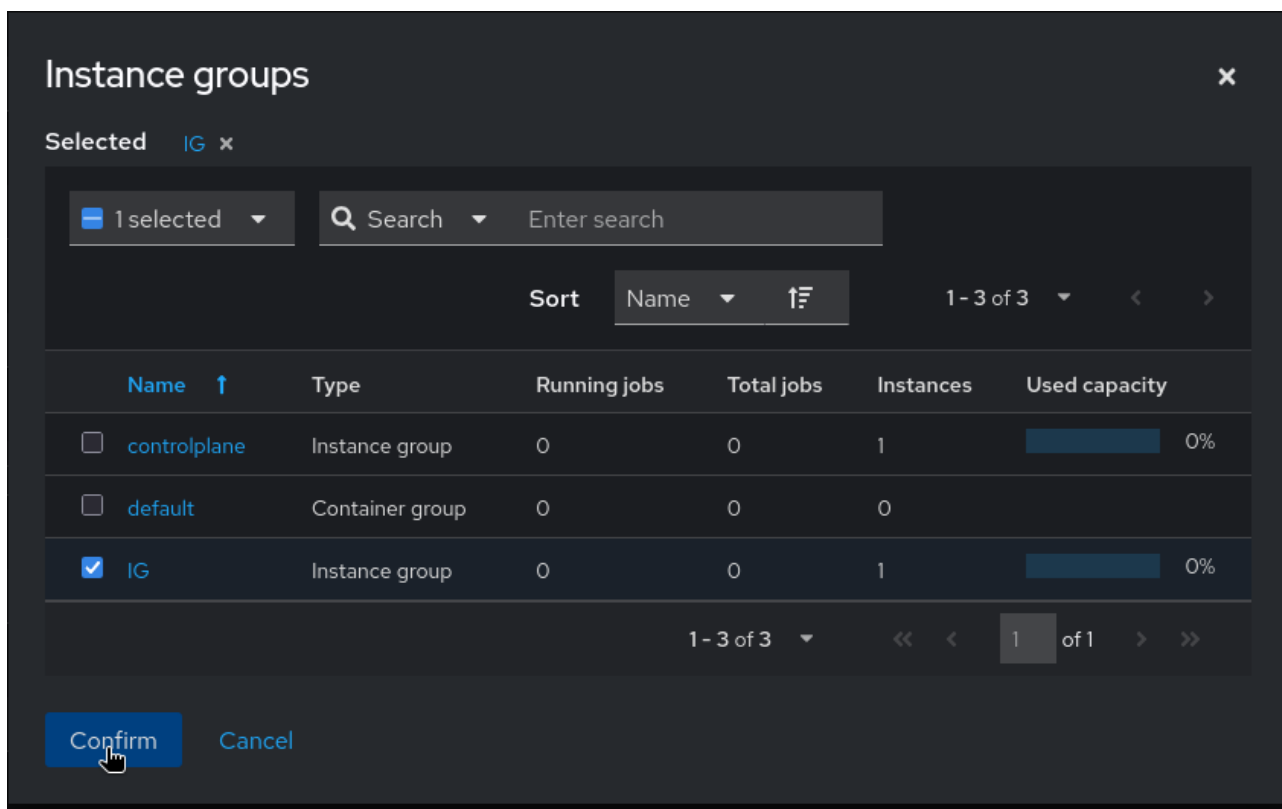
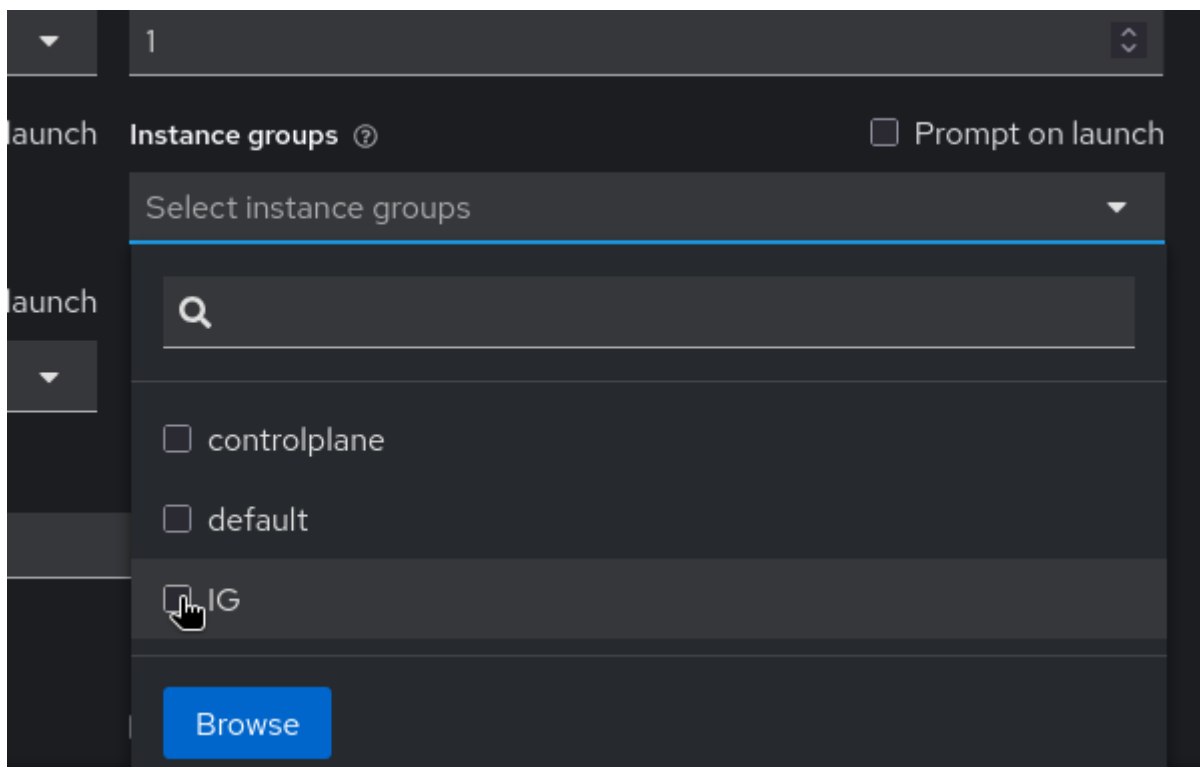


Part 2: Run a Job Using the Instance Group

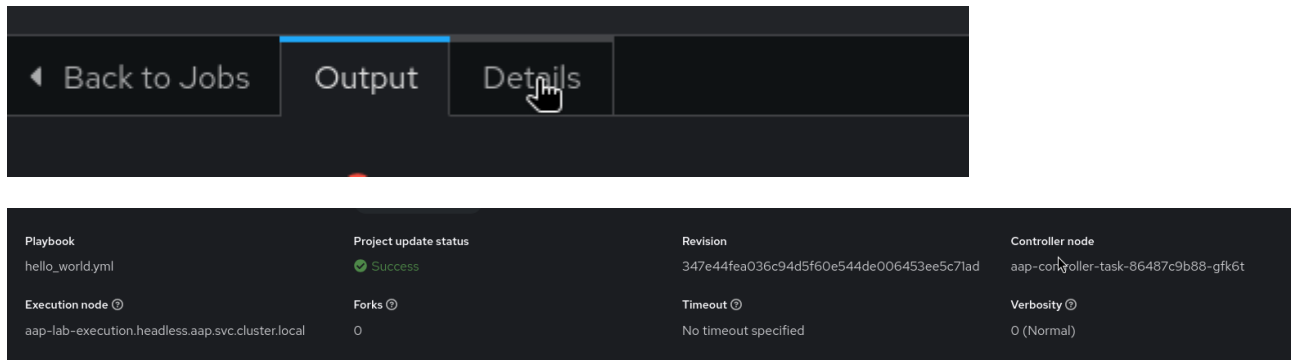
1. In the AAP UI, navigate to **Automation Execution > Templates**.
2. Click the **edit** (pencil) icon on the Demo Job Template to open its settings.



3. Click **Instance groups** and select **IG**, then save the template.

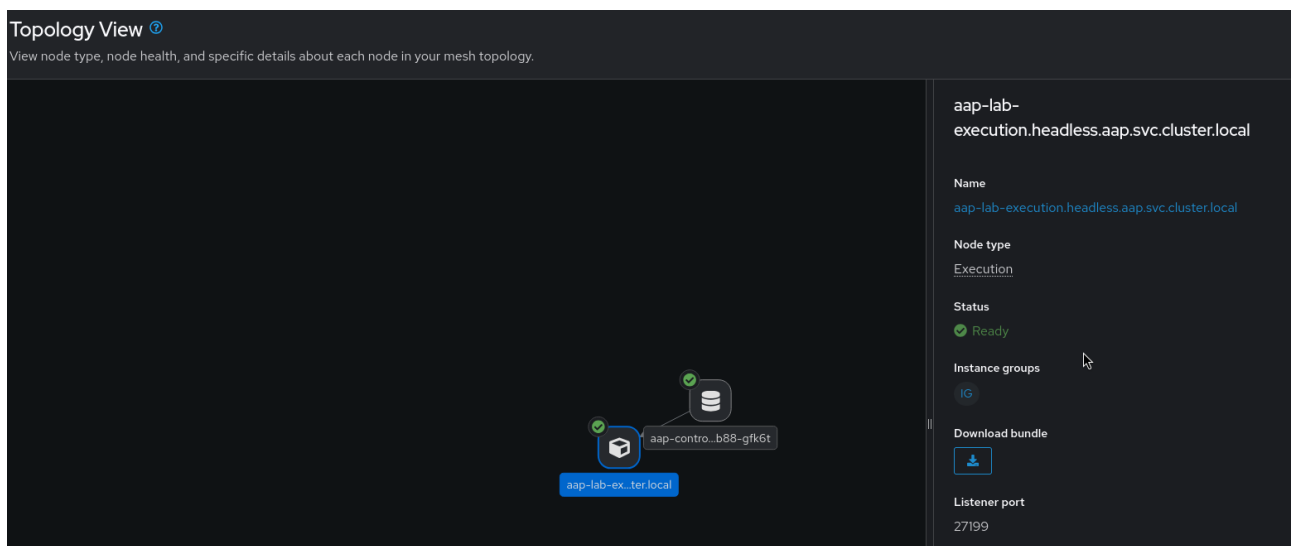


4. Launch the **Demo Job Template**.
5. In the AAP UI navigate to **Automation Execution > Jobs**, select the running job and check the **Details** section to verify it is executing via the Instance group via the execution node.



Part 3: View the Topology

1. In the AAP UI, navigate to **Automation Execution > Infrastructure > Topology View**.
2. Verify the graphical representation shows your Instance Group connected to the Control Plane.
3. Refer to the **Legend** to identify node types and link statuses.



Calculate Sizing Requirements for an OpenShift Deployment

Having the right amount of resources for Ansible Automation Platform helps make the platform reliable and reduces cost. In an operator-based deployment on OpenShift, sizing is expressed as Pod resource **requests** and **limits** rather than VM specifications. The high-level points that should be considered during capacity planning are:

- Characterizing the workloads.
- Reviewing the capabilities of different component pods.
- Planning the deployment based on the requirements of the workload.

The same isolated factors apply as in a VM-based deployment:

- Number of managed hosts controlled by AAP.
- Jobs or tasks running per hour per host.
- Maximum number of concurrent jobs to support.
- Maximum number of forks set on jobs.
- Maximum API requests per second.
- Target Pod CPU and memory sizes.

Automation Controller Pods

The AAP Operator deploys automation controller as multiple pods. The two primary workload pods are:

- **web**: Handles the UI and REST API. Scales horizontally — add more web pod replicas to handle more concurrent API requests.
- **task**: Dispatches jobs, processes job events, and updates the database. CPU and memory on this pod determine control capacity (events per second).

Scale the **task** pod vertically (increase CPU and memory) to increase event processing throughput. Scale **web** pods horizontally to increase API concurrency.

Recommended ratio for the **task** pod: **1 CPU to 4 GB RAM**. Add CPU together with memory even if CPU is not a bottleneck — during high memory utilization the queue fills with unprocessed events, and higher CPU helps drain the queue faster.

Container Groups (Execution Capacity on OCP)

Container Groups run jobs as ephemeral pods. Execution capacity is determined by the resource limits set on each pod in the **Pod spec override**, not by a node's total memory. Key points:

- Forks are set at the job template level and passed directly to the container.
- Container groups do **not** use the fork capacity algorithm (memory / 100 MB) used by execution nodes.
- Use **LimitRange** objects in your namespace to set namespace-wide pod resource limits.
- Set **resources.requests** and **resources.limits** in the pod spec override for fine-grained control.

```
resources:
  requests:
    cpu: 250m
    memory: 100Mi
  limits:
    cpu: "2"
    memory: 2Gi
```

Remote Execution Nodes (RHEL VMs peered to OCP)

Remote execution nodes are RHEL VMs that extend the mesh beyond the OCP cluster. Their capacity is calculated the same way as in a VM-based deployment:

Memory Relative Capacity

`mem_capacity` is the maximum memory needed by one fork. The default value is 100 MB. 2 GB of memory is reserved for AAP services.

Example: 16 GB of available node memory $\rightarrow (16384 - 2048) / 100 \approx 140$ forks

CPU Relative Capacity

The baseline value is 1 CPU = 4 forks.

4 CPUs $\times$ 4 forks per CPU = 16 forks

The `capacity_adjustment` value (0 to 1) determines whether fork count is closer to the CPU-bound or memory-bound limit. At 0.5:

Capacity = $[140 \text{ forks memory} \times 0.50] + [16 \text{ forks CPU} \times 0.50]$
= $70 + 8 = 78$ forks

Automation Hub (Private Automation Hub)

Private Automation Hub provides Execution Environments and collections, so it has modest CPU and memory needs. Storage sizing dominates:

- Number of Execution Environments stored.
- Size of each Execution Environment.
- Number of versions of each Execution Environment.

In an OCP deployment, storage is provided by Persistent Volume Claims (PVCs). Ensure the selected StorageClass supports `ReadWriteMany` if multiple hub pods are needed, or `ReadWriteOnce` for single-pod deployments.

Event-Driven Ansible Controller (AutomationEDA)

Event-Driven Ansible workloads are sized based on:

- Number of simultaneous rulebook activations.
- Number of events received per minute.

In an OCP deployment, rulebook activation limits are configured via the `AutomationEDA` custom resource spec rather than a file-based settings file. The default is 12 rulebook activations per node

and 24 activations total. Memory per activation container is set as a Pod resource limit in the CR — the equivalent of the `PODMAN_MEM_LIMIT` setting in non-OCP deployments.

Database (PostgreSQL)

The AAP Operator integrates with managed PostgreSQL (via Crunchy Data operator or an external database). Database Pod sizing should match or exceed the higher of the Control Plane or Execution Plane resource requirements:

- Database storage: see [sizing example](#) for formulas.
- Database RAM and CPU: align with the `task` pod sizing.

Example: Calculating Sizing for an OpenShift Deployment

At this point we should have a good idea how to calculate capacity and what factors to consider. Let's go through the same planning exercise applied to an OCP deployment.

For this example, the cluster must support the following capacity:

- 500 managed hosts
- 1,000 tasks per hour per host (16 tasks per minute per host)
- 10 concurrent jobs
- Forks set to 5 on playbooks (the default)
- The average event size is 1 KB

Target pod sizes: 4 vCPU and 16 GB RAM each.

Execution Capacity Calculation

Execution Capacity = (Number of jobs × forks value) + (Number of jobs × 1 base task impact)

$(10 \text{ jobs} \times 5 \text{ forks}) + (10 \text{ jobs} \times 1) = 60 \text{ execution capacity units}$

For a Container Group, this means each pod spec should allow at least 5 forks. Set the fork limit at the job template level. No special node sizing formula applies — forks are passed directly to the pod.

For a remote execution node peered to OCP:

$16 \text{ GB RAM} \rightarrow (16384 - 2048) / 100 = 140 \text{ forks available}$
 $140 \text{ forks} > 60 \text{ required} \rightarrow 1 \text{ execution node is sufficient}$

Two nodes are recommended for redundancy.

Control Capacity Calculation

Total tasks/min = 500 managed hosts x 16 tasks/min = 8,000 tasks/min = 133 tasks/sec

Assuming 6 events per task:

133 tasks/sec x 6 events/task = 798 events/sec
798 events x 1 KB/event = 798 KB/sec of event log data

The automation controller **task** pod must be able to process approximately 798 events per second. The default automation controller throughput is ~400 events/second per **task** pod replica.

Therefore, for this workload **2 task pod replicas** are recommended. Set this in the **AutomationController** CR:

```
apiVersion: automationcontroller.ansible.com/v1beta1
kind: AutomationController
metadata:
  name: aap-controller
  namespace: aap
spec:
  replicas: 2
  task_resource_requirements:
    requests:
      cpu: "4"
      memory: 16Gi
    limits:
      cpu: "4"
      memory: 16Gi
  web_resource_requirements:
    requests:
      cpu: "2"
      memory: 8Gi
    limits:
      cpu: "2"
      memory: 8Gi
```



In the **AutomationController** CR, **replicas** controls the number of automation controller instances (task pods). The **web** pod scales independently and handles API and UI requests. Vertical scaling (more CPU/RAM) on the task pod is the primary lever for event throughput. Always set CPU **requests** equal to CPU **limits** to prevent CFS throttling — as shown in the YAML above.

Database Sizing

Database size for facts = Facts size per host x number of hosts / 1024

$$= 50 \text{ KB} \times 500 / 1024 \approx 24 \text{ MB}$$

Database size for jobs = hosts x jobs/host/day x retention days x events x event size / 1024

$$= 500 \times 24 \times 30 \times 6 \times 1 \text{ KB} / 1024 \approx 2,109 \text{ MB} \approx 2 \text{ GB (30-day retention)}$$

For an OCP deployment, provision a PVC for PostgreSQL sized to at least **50 GB** to account for growth and WAL logs.

Summary

To summarize, for this workload on OpenShift:

- **Automation Controller:** 2 replicas, 4 vCPU / 16 GB RAM per task pod; 2 vCPU / 8 GB RAM per web pod.
- **Container Group (in-cluster execution):** Pod spec with resources appropriate to the fork count (250m–2 CPU, 100Mi–2Gi per worker pod).
- **Remote Execution Node (if needed):** 1–2 RHEL VMs, 4 vCPU / 16 GB RAM each.
- **PostgreSQL PVC:** 50 GB minimum.



For Automation Hub and EDA, start with the default operator sizing and scale based on observed storage and activation usage.

Performance Considerations for OpenShift Deployments

In the previous section we discussed how to calculate node size. The following default parameters and OCP-specific factors help with planning control and execution capacity.



The Tunning of the parameters needs careful consideration and should be done only after measuring the actual workload and performance of the system. The default values are based on typical workloads and may not be optimal for all environments.

Default Parameters

The following parameters are unlikely to need changes unless you have a specific reason and measured value:

Parameter	Default value
Facts size per host	50 KB
Size per event	1 KB
Events per task	6–10

Parameter	Default value
Memory per fork (execution nodes)	100 MB
Forks per CPU (execution nodes)	4
Capacity Adjustment	1.0 (select highest value)
Execution Environment average size	450 MB
Automation Controller event throughput (per task pod)	~400 events/sec
Automation Controller RAM per event fork	10 KB
Automation Controller CPU per event fork	0.0001
Concurrent API calls per controller web pod	100 (up to 300 allowed)
Controller/execution base job capacity per job run	1



In an OCP deployment, "Memory per fork" applies to remote execution nodes only. Container Group pods do not use the fork capacity formula — set fork limits directly in the pod spec or at the job template level.

Factors That Vary Across Environments

- Number of Ansible managed hosts/nodes.
- Number of jobs per host per day.
- Tasks run per playbook or job including imported roles/tasks/playbooks.
- Whether jobs run for a specific time period or 24×7.

CPU Throttling Prevention

In OpenShift Container Platform, setting a CPU **limit** without an equal CPU **request** causes the kernel CFS scheduler to throttle the container even when the node has spare CPU capacity. This throttling directly increases API latency and slows job event processing.

Rule: always set CPU **requests equal to CPU **limits** for all AAP pods.**

```
task_resource_requirements:
  requests:
    cpu: "4"
    memory: 16Gi
  limits:
    cpu: "4"          # must equal requests.cpu to avoid CFS throttling
    memory: 16Gi
```

Monitor CPU throttling with the Prometheus metric:


```
container_cpu_cfs_throttled_seconds_total
```

If this metric is non-zero and rising, increase the CPU **request** and the matching **limit** on the affected pod. Do not raise only the limit — that is the source of the throttling.

Manage Live Event Streams to the UI

The automation controller UI receives real-time job updates over WebSocket connections. Under heavy concurrent workloads, live event streaming increases load on the controller service.

Disable live streaming

Set **UI_LIVE_UPDATES_ENABLED** to **false** in the automation controller configuration. When disabled, the job detail page no longer auto-updates as events arrive — users must refresh manually to see current status.

```
extra_settings:
  - setting: UI_LIVE_UPDATES_ENABLED
    value: "false"
```

Tune event rate and size

The following settings control the volume of event data sent to the UI. Reducing these values lowers controller load at the cost of less granular live output:

Setting	Default	Effect
EVENT_STDOUT_MAX_BYTES_DISPLAY	1024	Maximum bytes of stdout displayed per event in the UI
MAX_WEBSOCKET_EVENT_RATE	30	Maximum events per second sent over WebSocket per job
JOB_EVENT_BUFFER_SECONDS	1	Time events are buffered before being sent to the UI

Job Type Impact on Capacity

Not all jobs consume the same controller capacity. Plan your sizing based on the actual workload mix:

Job type	Capacity impact	Notes
Standard jobs	1 fork = 1 capacity unit	The baseline — most playbook runs
Workflow jobs	Low on controller	Orchestrate child jobs; controller overhead is minimal but each child job consumes its own capacity

Job type	Capacity impact	Notes
Sliced jobs	1 unit per slice	A sliced job with 5 slices consumes 5 capacity units simultaneously
Bulk jobs	Sum of all child jobs	High total capacity; schedule during low-traffic windows
System jobs	Variable	Fact cleanup, project sync, inventory sync — compete with user jobs for task pod capacity

Project synchronizations and inventory syncs run as system jobs. During heavy project sync periods, reduce the number of concurrent jobs or schedule syncs during off-peak hours to avoid starving user workloads.

OpenShift-Specific Performance Considerations

Horizontal Pod Autoscaling

The `web` pod handles all UI and REST API traffic and scales horizontally — increase the `replicas` field in the `AutomationController` CR to add more web pod replicas. The `task` pod processes job events and scales vertically — increase its CPU and memory requests and limits to raise event throughput. When scaling `replicas`, also raise `web_resource_requirements` if API latency is the bottleneck.

Resource Quotas and LimitRanges

Ensure the AAP namespace does not have a `ResourceQuota` or `LimitRange` that caps pod memory below your sizing requirements. Quota exhaustion causes pod scheduling failures that appear as `Pending` pod status.

```
oc describe quota -n aap
oc describe limitrange -n aap
```

StorageClass and IOPS

Database and project storage performance on OCP depends on the underlying `StorageClass`. For production deployments, use a `StorageClass` backed by block storage. Provision at least 3,000 IOPS for the PostgreSQL PVC. Avoid `ReadWriteOnce` PVCs if you plan to run multiple hub or controller replicas — those require `ReadWriteMany`.

WebSocket Load Balancing

When running multiple `web` pod replicas, WebSocket connections require sticky sessions — the same client must be routed to the same pod for the life of the connection. Configure source-based load balancing on the OpenShift Route:

```
oc annotate route aap-controller \
haproxy.router.openshift.io/balance=source
```

The **source** algorithm pins a client to the same backend pod based on source IP, maintaining WebSocket session continuity across reconnects.

ImagePullSecrets for Secured Registries

If your Execution Environments are stored in a private registry other than the default **registry.redhat.io**, configure an **imagePullSecret** in the **AutomationController** CR or in the pod spec override of the Container Group:

```
oc create secret docker-registry my-registry-secret \
  --docker-server=<registry-host> \
  --docker-username=<username> \
  --docker-password=<password> \
  -n aap
```

```
spec:
  image_pull_secrets:
    - name: my-registry-secret
```

Container Group Pod Overhead

Each Container Group job pod incurs Kubernetes scheduling overhead (approximately 2–5 seconds for pod start). For very short-lived playbooks, a persistent remote execution node provides better throughput than Container Groups, because the receptor connection is always up and there is no scheduling delay per job.

Key Performance Indicators

Monitor these metrics to determine when scaling or tuning is needed:

Indicator	Signal	Action
High API latency	Response time > 2 seconds for UI or API calls	Scale web pod replicas; check CPU throttling; check database connection pool
High CPU utilization	Task pod CPU sustained > 80%	Increase task pod CPU requests and limits equally to avoid throttling
502 Bad Gateway	Returned by platform gateway	Scale gateway replicas; check backend pod health
503 Service Unavailable	Returned by automation controller	Scale web pod replicas; check if task pod is overloaded
429 Too Many Requests	Returned by platform gateway	Rate-limiting triggered; use token-based API access; reduce polling frequency in clients

Indicator	Signal	Action
<code>container_cpu_cfs_throttled_seconds_total</code> rising	Prometheus metric	Set CPU <code>requests</code> equal to CPU <code>limits</code> on the affected pod

OpenShift Route Timeout

Long-running API requests — such as job launch polling or large inventory syncs — can exceed the default HAProxy Route timeout of 30 seconds, causing the client to receive a 504 error even though the backend is still working. Increase the timeout with the following annotation on each AAP Route:

```
oc annotate route aap-controller \
  haproxy.router.openshift.io/timeout=300s
```

Apply to each route that clients access (controller, hub, EDA gateway). Clients should also configure a matching or longer timeout on their side to avoid premature disconnection.

Helpful Formulas

Total Jobs = (Number of Hosts × Jobs per Host per Day) ÷ Hosts Required per Job

Job Duration = Job Duration per Host × CEILING(Hosts per Job ÷ Parallel Forks)

Workload Utilization = (Jobs per Day × Job Duration) ÷ Automation Hours per Day

Polling Load = (Jobs per Day × Job Duration) ÷ Polling Interval

API Utilization = (API Calls per Day × API Call Duration) ÷ Automation Hours per Day

Database size for facts = Facts size per host × number of hosts / 1024

Database size for jobs = number of hosts × jobs per host per day × number of days
to keep the data × number of events × event size / 1024

Database RAM and CPU should be sized similarly to the `task` pod or Execution Plane node, whichever is higher.

Troubleshooting AAP Installation Issues on OpenShift

In an operator-based deployment, the AAP Operator reconciles custom resources and manages pod lifecycle. Most installation and configuration issues surface as failed operator reconciliation conditions, pods stuck in `Pending` or `CrashLoopBackOff`, or custom resources that never reach a `Running` state. The basic rule of thumb: check the operator conditions first, then the pod logs.

- Login to the `{bastion_public_hostname}` using the command `{bastion_ssh_command}` and perform the following steps to know how to troubleshoot the AAP installation issues on OpenShift.

- Login to the OpenShift cluster using the command `oc login -u admin -p {openshift_cluster_admin_password} {openshift_api_server_url}`.

Step 1: Check the Operator CSV Status

Verify the AAP Operator is installed and healthy:

```
oc get csv -n aap
```

The **PHASE** column should show **Succeeded**. If it shows **Failed** or **Pending**, describe the CSV for details:

```
oc describe csv <csv-name> -n aap
```

Step 2: Check Custom Resource Conditions

Check the status of your **AutomationController** CR:

```
oc describe automationcontroller aap-controller -n aap
```

Look at the **Status.Conditions** section. Common conditions include:

- **Running**: Controller is up and healthy.
- **Failure**: The operator encountered an error — the **message** field describes the cause.
- **Degraded**: Some components are not ready.

Similarly for other CRs:

```
oc describe automationhub aap-hub -n aap
oc describe automationeda aap-eda -n aap
```

Step 3: Check Pod Status and Logs

List all pods in the AAP namespace:

```
oc get pods -n aap
```

For pods in **Pending** state, check events:

```
oc describe pod <pod-name> -n aap
```

For pods in **CrashLoopBackOff** or with errors, check logs:

```
oc logs -n aap deployment/aap-controller-web
oc logs -n aap deployment/aap-controller-task
oc logs -n aap deployment/aap-controller-task --previous
```

Step 4: Increase Ansible Verbosity

To get more verbose output from the operator's internal automation, add `task_args` to the `AutomationController` CR spec:

```
spec:
  task_args:
    - "-vvv"
```

Apply the change and monitor the `task` pod logs.

Common OpenShift-Specific Issues

PVC not bound (wrong StorageClass)

Issue: PostgreSQL pod stays in `Pending`.

Fix: Check that the `StorageClass` specified in the CR exists and supports dynamic provisioning:

```
oc get pvc -n aap
oc get storageclass
```

Route not created (missing wildcard DNS)

Issue: AAP UI is unreachable.

Fix: Verify that your OCP cluster has a wildcard DNS entry for the router domain and that the `Route` resource was created:

```
oc get route -n aap
```

Image pull failures (missing pull secret)

Issue: Pods show `ImagePullBackOff`.

Fix: Create a pull secret for `registry.redhat.io` and link it to the namespace service account:

```
oc create secret docker-registry rh-registry-secret \
  --docker-server=registry.redhat.io \
  --docker-username=<rh-username> \
  --docker-password=<rh-password> \
  -n aap
oc secrets link default rh-registry-secret --for=pull -n aap
```

Resource quota exceeded

Issue: Pods are **Pending** with **exceeded quota** in events.

Fix: Check namespace quota and either increase it or reduce resource requests in the CR spec:

```
oc describe quota -n aap
```

Execution node not connecting to controller

Issue: Instance shows **Unavailable** in the Instances page.

Ensure port 27199 (default Receptor listener port) is open between the execution node and the OCP cluster ingress. Verify TLS certificates match by re-downloading and re-running the install bundle.

Summary

In this chapter we learned the following about deploying Automation Mesh on OpenShift Container Platform:

- In operator-based deployments there are **no Hybrid or Control nodes** — the Control Plane is made up of pods managed by the AAP Operator running inside the Kubernetes cluster.
- **Container Groups** are the in-cluster execution mechanism: jobs run as ephemeral pods provisioned on-demand, using a service account with RBAC permissions to create and manage pods.
- Remote execution and hop nodes are RHEL VMs peered to the OCP-hosted controller through Receptor; they are added dynamically via the AAP UI **Instances** page without re-running the installer.
- **Mesh Ingress** (**AutomationControllerMeshIngress** CR) enables pull-based connectivity for remote nodes in networks that forbid inbound connections.
- Sizing on OCP is expressed as Pod **resource requests and limits** in the **AutomationController** CR, rather than VM memory/CPU specifications.
- The 1 CPU = 4 forks and 100 MB per fork rules still apply to remote execution nodes; Container Group pods set forks at the job template level.
- Troubleshooting uses **oc describe automationcontroller**, **oc get pods**, and **oc logs** rather than manual RPM installer steps.
- Common OCP-specific issues include PVC not bound, missing pull secrets, resource quota exhaustion, and wildcard DNS misconfiguration.
- **CPU throttling**: always set CPU **requests** equal to CPU **limits** on AAP pods. Setting a limit without an equal request causes CFS throttling even when the node has spare capacity — monitor with **container_cpu_cfs_throttled_seconds_total**.
- Live event streaming to the UI can be disabled with **UI_LIVE_UPDATES_ENABLED=false** to reduce controller load under heavy workloads; event rate and buffer settings further tune the trade-off between live visibility and system load.
- Job types (standard, sliced, workflow, system) have different capacity impacts — size the task

pod based on your actual workload mix, not just concurrent job count.

- Use key performance indicators (API latency > 2s, sustained CPU > 80%, 502/503/429 error codes) to determine whether to scale web pods horizontally or the task pod vertically.
- Increase the OpenShift Route timeout ([haproxy.router.openshift.io/timeout](#)) on all AAP routes when clients report 504 errors on long-running operations.

Appendix

AAP 2.7 Documentation

- [Ansible Automation Platform 2.7 Documentation](#)
- [Administer Ansible Automation Platform 2.7](#)

Automation Mesh

- [Automation Mesh for Operator \(OCP\) Environments](#)
- [Automation Mesh for Containerized and VM Environments](#)

Installation

- [Containerized AAP Installation Guide](#)
- [Deploying AAP on OpenShift Container Platform](#)

Sizing and Capacity Planning

- [Red Hat Ansible Automation Platform Sizing Guide](#)

Performance Optimization

- [Optimize Ansible Automation Platform 2.7](#)